Inverse Kinematics and Workspace Analysis of X-Y Pedestal with Joint and Pointing Misalignment

Buğra Coşkun*

05.09.2023

Abstract

We present an inverse kinematics solution for X-Y pedestals with joints and pointing misalignment. We use vector analysis for optimal computational complexity of the inverse kinematics solution. We show how misalignment in the joints affect the workspace of the pedestal and present examples in Octave and C for the application of the solution.

Introduction

The 2R serial manipulator configurations are popular solutions for satellite ground stations with mechanical tracking capabilities. Most widely used is the altitude-azimuth configuration because of many advantages such as simplicity and cost-effectiveness. But with their simplicity comes the disadvantage of the zenith problem caused by an increase in needed azimuth velocities[1]. To combat the problem with tracking objects in high relative elavations the X-Y configuration is another popular option for ground terminals, In this configuration the joints are moving slower compared to altitude-azimuth configuration at higher elevations but a similar mechanical limitation arises that decreases the manipulator's effectiveness at relatively low elevations. As the need for low elevation tracking is quite rare the X-Y configuration usually happens to be an optimal solution for the pedestal choice[3]. Research into the kinematics and dynamics for the X-Y manipulator has been previously explored using D-H parameters for the inverse kinematics problem [2]. The synthesized solutions use trigonometric functions but the added misalignment properties increases the complexity of this method. We shall instead employ vector analysis approach to keep the complexity of the equations relatively tame.

1 Forward Kinematics

For the forward kinematics we first consider the rotation of the Y-joint, because the Y-joint axis is transformed by the X-joint rotation, considering

*E-mail: bugra.coskun@profen.com

this rotation first will preserve us from calculating the new axis of the Y-joint. This is an example of extrinsic rotation. The pedestal orientation vector, Y-joint axis and X-joint axis are denoted as $\hat{p}_0, \hat{w}_0, \hat{x}_0$ respectively.

$$\hat{w}_0 = \begin{bmatrix} \cos(\lambda) \\ \sin(\lambda) \\ 0 \end{bmatrix}, \; W_0 = \begin{bmatrix} 0 & -k_3 & k_2 \\ k_3 & 0 & -k_1 \\ -k_2 & k_1 & 0 \end{bmatrix} \; \therefore \; W_0 \vec{v} = \hat{w}_0 \times \vec{v}. \tag{1}$$

The rotation matrix in $SO(3)$ group around an arbitrary axis is given by the Rodrigues' rotation formula:

$$R_{\hat{w}_0}(\phi) = I + (\sin\phi) W_0 + (1 - \cos\phi) W_0^2, \tag{2}$$

The standard rotation matrix around the $\hat{x}$ axis is in the form:

$$R_x(\psi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\psi & -\sin\psi \\ 0 & \sin\psi & \cos\psi \end{bmatrix}, \tag{3}$$

Now that we have the rotation matrices that we require, the forward kinematic equation becomes:

$$\hat{l} = R_x(\psi) R_{w_0}(\phi) \hat{p}_0, \tag{4}$$

The resulting line-of-sight vector is in the pedestal frame, to get the orientation in the NWU(north-west-up) frame we matrix multiply the LOS vector with the the inverse of the NWU rotation matrix defined in the pedestal basis. For further details on the NWU matrix and frame see Appendix B.

$$\hat{l}_N = R_N^{-1} \hat{l}. \tag{5}$$

The resulting azimuth and elevation degrees is given by the cartesian to spherical conversion. For our applications we do not need the radial distance and it is more convenient to use the elevational angle instead of the polar angle. We define the function and it's inverse for these conversions as such:

$$\Lambda : \mathbb{R}^2 \mapsto \mathbb{R}^3, \; \phi \in (-\pi, \pi], \; \theta \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right], \tag{6}$$

$$\Lambda\left(\begin{bmatrix} \phi \\ \theta \end{bmatrix}\right) = \begin{bmatrix} \cos(\theta)\cos(\phi) \\ \cos(\theta)\sin(\phi) \\ \sin(\theta) \end{bmatrix} \text{ and } \Lambda^{-1}(\hat{v}) = \begin{bmatrix} \operatorname{atan2}(v_2, v_1) \\ \sin^{-1}(v_3) \end{bmatrix}, \tag{7}$$

$$\therefore \begin{bmatrix} -\alpha \\ \beta \end{bmatrix} = \Lambda^{-1}(\vec{l}_N). \tag{8}$$

Where α and β are the azimuth and elevation degrees of the resulting pedestal orientation. The azimuth angle is taken the negative of because azimuth degree is CW positive contrary to the conventional spherical coordinates.

2 Inverse Kinematics

For a pedestal orientation we are most often going to be supplied with an azimuth and elevation angles, thus we first calculate the orientation vector using the function from Equation (7).

$$\hat{l} = \Lambda\left(\begin{bmatrix} -\alpha \\ \beta \end{bmatrix}\right) = \begin{bmatrix} l_1 \\ l_2 \\ l_3 \end{bmatrix} \tag{9}$$

For the inverse kinematics we need to know what Y-joint angle brings the target vector into a position that will reach the target orientation when subjected to a rotation about the X-joint.

$$\mathbb{V} = \{\vec{v} \in \mathbb{R}^3 : ||\vec{v}|| = 1\} \tag{10}$$

$$c \in [-1, 1] \to \mathbb{S}_{\hat{w}}(c) = \{\hat{v} : \hat{v} \cdot \hat{w} = c\}. \tag{11}$$

$$\forall \hat{v} \in \mathbb{S}_{\hat{w}}(c), \forall \theta \in (-\pi, \pi] \to R_{\hat{w}}(\theta)\hat{v} \in \mathbb{S}_{\hat{w}}(c) \because R_{\hat{w}}(\theta) \in SO(3). \tag{12}$$

$$\hat{l} \in \mathbb{S}_{\hat{x}}(\hat{l} \cdot \hat{x}) \therefore R_{\hat{x}}(\psi)R_{\hat{w}_0}(\phi)\hat{p}_0 = \hat{l} \Leftrightarrow R_{\hat{w}_0}(\phi)\hat{p}_0 \in \mathbb{S}_{\hat{x}}(\hat{l} \cdot \hat{x}) \tag{13}$$

Acknowledgements

The author is thankful to Mustafa Çelik, Cem Doğan and Kadir Koca for their guidance on X-Y pedestals as well as to Merve Güvenç for the inspiring conversations and tips for structuring this paper.

References

[1] Tongbo Ji et al. "Analysis of tracking performance for altitude-azimuth pedestal near the zenith". In: *Fifth International Symposium on Instrumentation and Control Technology*. Ed. by Guangjun Zhang, Huijie Zhao, and Zhongyu Wang. Vol. 5253. International Society for Optics and Photonics. SPIE, 2003, pp. 398–401. DOI: 10.1117/12.521401. URL: https://doi.org/10.1117/12.521401.

[2] Nafiseh Mazaheri Tehrani et al. "Full extracting kinematic and dynamic equations of X/Y pedestal with velocity analysis". In: *2014 CACS International Automatic Control Conference (CACS 2014)*. 2014, pp. 215–221. DOI: 10.1109/CACS.2014.7097190.

[3] Keith Willey. "Selecting a pedestal for tracking LEO satellites at Ka band". In: *Microwave journal* 43.4 (2000), pp. 118–118.

Appendix A: North-West-Up Frame

Appendix B: Octave Code - Inverse Kinematics

```
args_cell = argv();
args_mat = str2double(args_cell);

function vect = spherical2cartesian(sp)
    % Map horizontal coordinates to cartesian coordinates
    % Warning: In a ned system elevation=-theta
    % Spherical coordinates to cartesian
    % sp = [azimuthal ccw + , elevation, distance]
    vect = [[ sp(3)*cos(sp(2))*cos(sp(1)) ];
            [ sp(3)*cos(sp(2))*sin(sp(1)) ];
            [ sp(3)*sin(sp(2)) ]];
end

function xy = Xyink(Azimuth_D, Elevation_D, Orthogonality_Error)
    ld = pi/2 + Orthogonality_Error;
    % We need to first multiply azimuth with -1 to fix for NED to NWU
    Offset = 90; % Degrees
    ModifiedAzimuth = mod(360 - (Azimuth_D + Offset), 360);
    SphericalP = deg2rad(ModifiedAzimuth);
    SphericalT = deg2rad(Elevation_D);
    losv = spherical2cartesian([SphericalP,SphericalT,1]);
    % Quadratic form to get the double roots of the X joint
    cotld = cos(ld)/sin(ld);
    nomp0 = -(losv(1)*cotld);
    nomp1 = sqrt( losv(1)^2 * cotld^2 - losv(2)^2 - losv(3)^2 );
    nom0 = nomp0 + nomp1;
    nom1 = nomp0 - nomp1;
    den = losv(2) - i * losv(3);

    PSI0 = mod(real(-i*log(nom0/den)), 2*pi);
    PSI1 = mod(real(-i*log(nom1/den)), 2*pi);

    % Check which of the X roots are within mechanical boundaries of the system
    PSI0 = PSI0 - 2*pi*( PSI0 > pi );
    PSI1 = PSI1 - 2*pi*( PSI1 > pi );

    if PSI0 < pi/2 && PSI0 > -pi/2
        PSI = PSI0;
    elseif PSI1 < pi/2 && PSI1 > -pi/2
        PSI = PSI1;
    else
        fprintf("FATAL ERROR: Out of workspace at < %d , %d > \n",...
            Azimuth_D, Elevation_D)
        if PSI0 < PSI1
            PSI = PSI0;
        elseif
            PSI = PSI1;
        end
    end

    % ---------------------------------------------------------------------- Y ANGLE
    % Rotation the X motor does to the Y

```

```
    Rx = [ 1 0 0 ; 0 cos(PSI) -sin(PSI) ; 0 sin(PSI) cos(PSI)];
    Zhat = [0 ; 0 ; 1];
    Tipv = Rx * Zhat;

    K0v = [ cos(ld) ; sin(ld) ; 0 ];

    % Rotated Y motor joint vector
    Kv = Rx * K0v;

    nTipLos = cross(Tipv, losv);
    TipLos = nTipLos / norm(nTipLos);
    unnormSign = dot(TipLos, Kv);
    % This is a bit weird the sign seems to be reverse of what its supposed to
    % be, need to look into it
    if unnormSign < 0
        phiSign = 1;
    elseif unnormSign > 0
        phiSign = -1;
    else
        phiSign = 0;
    end
    PHI = phiSign * acos( dot(Tipv, losv) );

    xy = rad2deg([ PSI ; PHI ]);
end

xyinkResult = Xyink(args_mat(1), args_mat(2), 0)
```